L13 – Encoding and BoW

Data Mining
Dr Andrea Visentin

Thanks to Dr Alejandro Arbelaez, Dr Derek Bridge
And to Flaticon.com

Variables

A Variable

- Is a **characteristic** of **an object** we seek to measure, count, enquire, observe, or compute
- Characteristic can take on different values based on **chance (or probability)**

An object can be:

Person

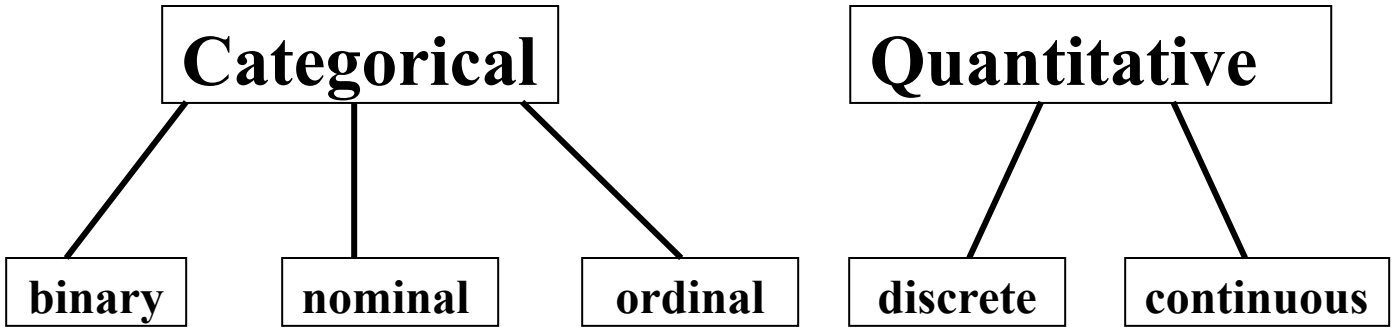
Laptop

Mobile phone

Terminology

- Features, variables, attributes, X
  - Input of your model
- Label, target, Y
  - Output/decision of your model
- The columns of a table contain features and labels
- The rows are the instances/objects/entries

Classification of Features



2 categories +

more categories +

order matters +

numerical +

uninterrupted

Types of Variables

- Quantitative variables:
  - Always numeric
  - Can be any number, positive or negative
  - Examples: age in years, weight, blood pressure readings, temperature, concentrations of pollutants and other measurements
- Categorical variables:
  - Information that can be sorted into categories
  - Types of categorical variables – ordinal, nominal and dichotomous (binary)

Hamming Distance Metric

- Allows us to deal with problems that have features that are discrete rather than continuous-valued
- The value 0 is assigned for each feature where both cases have the same value, 1 for each where they are different.

$$D(X_i, X_j) = \sum_{r=1}^{n} \begin{cases} 1 & if \quad a_r(X_i) \,!= a_r(X_j) \\ 0 & if \quad a_r(X_i) == a_r(X_j) \end{cases}$$

ID	Outlook	Temp	Hum	Windy	Play?
A	sunny	hot	high	false	no
B	sunny	hot	high	true	no
C	overcast	hot	high	false	yes

Hamming Distance Metric

- Allows us to deal with problems that have features that are discrete rather than continuous-valued
- The value 0 is assigned for each feature where both cases have the same value, 1 for each where they are different.

$$D(X_i, X_j) = \sum_{r=1}^{n} \begin{cases} 1 & if \quad a_r(X_i) \,!= a_r(X_j) \\ 0 & if \quad a_r(X_i) == a_r(X_j) \end{cases}$$

ID	Outlook	Temp	Hum	Windy	Play?
A	sunny	hot	high	false	no
B	sunny	hot	high	true	no
C	overcast	hot	high	false	yes

What is distance between B and C?

Outlook (1) + Temp(0) + Hum(0) + Windy(1) = 2

Categorical Variables: Ordinal Variables

- Ordinal variable—a categorical variable with some intrinsic order or numeric value
- Examples of ordinal variables:
  - Education (no high school degree, HS degree, some college, college degree)
  - Agreement (strongly disagree, disagree, neutral, agree, strongly agree)
  - Rating (excellent, good, fair, poor)
  - Frequency (always, often, sometimes, never)
  - Any other scale ("On a scale of 1 to 5...")

Encoding Categorical Features Ordinal Variables

- To make sure that our learning algorithms interpret the ordinal features correctly we need to convert the **categorical string values into integers**.
- Unfortunately, as the ordering of ordinal values is typically related to the domain and as such there is no automatic mechanism of encoding this information.
- Therefore, you need to **specify the mapping manually**, which can be time consuming. In the example on the next slide we specify the mapping for the ordinal feature Grades.

Sample dataset

```
import pandas as pd

seriesA = pd.Series(['A', 'C', 'B'])
seriesB = pd.Series([21, 18, 19])
seriesC = pd.Series([4, 1, 1])
seriesD = pd.Series(['Computing', 'Biology', 'Biology'])

df = pd.DataFrame({'Grade' : seriesA, 'Age' : seriesB,
'DegreeYear' : seriesC, 'Department' : seriesD})
print (df)
```

Sample dataset

```
import pandas as pd

seriesA = pd.Series(['A', 'C', 'B'])
seriesB = pd.Series([21, 18, 19])
seriesC = pd.Series([4, 1, 1])
seriesD = pd.Series(['Computing', 'Biology', 'Biology'])

df = pd.DataFrame({'Grade' : seriesA, 'Age' : seriesB,
'DegreeYear' : seriesC, 'Department' : seriesD})
print (df)
```

	Grade	Age	DegreeYear	Department
0	A	21	4	Computing
1	C	18	1	Biology
2	B	19	1	Biology

Category Mapping

```
grade_mapping = {'F':0, 'D':1, 'C':2, 'B':3, 'A':4}
df['Grade'] = df['Grade'].map(grade_mapping)
print (df)
```

```
   Grade  Age  DegreeYear Department
0      4   21           4  Computing
1      2   18           1    Biology
2      3   19           1    Biology
```

Category Mapping

- Notice we create a dictionary that provides a mapping from letter grades to numerical values.
- We then provide call the map method for the Grade column, which takes the dictionary as an argument.

Category Mapping - Scikit learn

- We could directly encode nominal values into integer values using the LabelEncoder from Scikitlearn.
- https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html

Category Mapping - Scikit learn

```
from sklearn.preprocessing import LabelEncoder
# We continue using the dataframe df from the previous slides

class_le = LabelEncoder()
df['Department'] = class_le.fit_transform(df['Department'].values)
print (df)
```

	Grade	Age	DegreeYear	Department
0	A	21	4	1
1	C	18	1	0
2	B	19	1	0

Category Mapping - Scikit learn

```
import pandas as pd

seriesA = pd.Series(['A', 'C', 'B'])
seriesB = pd.Series([21, 18, 19])
seriesC = pd.Series([4, 1, 1])
seriesD = pd.Series(['Computing', 'Biology', 'Chemistry'])

df = pd.DataFrame({'Grade' : seriesA, 'Age' : seriesB, 'DegreeYear'
: seriesC, 'Department' : seriesD})
print(df)

from sklearn.preprocessing import LabelEncoder
# We continue using the dataframe df from the previous slides
class_le = LabelEncoder()
df['Department'] = class_le.fit_transform(df['Department'].values)
print(df)
```

Category Mapping - Scikit learn

```
import pandas as pd

seriesA = pd.Series(['A', 'C', 'B'])
seriesB = pd.Series([21, 18, 19])
seriesC = pd.Series([4, 1, 1])
seriesD = pd.Series(['Computing', 'Biology', 'Chemistry'])

df = pd.DataFrame({'Grade' : seriesA, 'Age' : seriesB, 'DegreeYear'
: seriesC, 'Department' : seriesD})
print(df)
from sklearn.pr
# We continue u
class_le = Labe
df['Department'
print(df)
```

```
   Grade  Age  DegreeYear Department
0      A   21           4  Computing
1      C   18           1    Biology
2      B   19           1  Chemistry
```

```
   Grade  Age  DegreeYear  Department
0      A   21           4           2
1      C   18           1           0
2      B   19           1           1
```

slides

'].values)

Category Mapping - Scikit learn

- Although the Department feature has no specific ordering, a learning algorithm will now assumes that the Computing value is **larger** than the Chemistry value, which is turn is **larger** than the Biology value.

- Although the assumption is incorrect, the algorithm could still produce useful results.

	Grade	Age	DegreeYear	Department
0	A	21	4	Computing
1	C	18	1	Biology
2	B	19	1	Chemistry

	Grade	Age	DegreeYear	Department
0	A	21	4	2
1	C	18	1	0
2	B	19	1	1

Categorical Variables: Nominal Variables

- Nominal variable – a categorical variable without an intrinsic order
- Examples of nominal variables:
  - Where a person lives in the U.S. (Northeast, South, Midwest, etc.)
  - Sex (male, female)
  - Nationality (American, Mexican, French)
  - Race/ethnicity (African American, Hispanic, White, Asian American)
  - Favorite pet (dog, cat, fish, snake)

One-hot Encoding

- A common way of addressing this problem is to a popular technique referred as **'one-hot encoding'**
- This technique will create new dummy features for each unique value in the nominal feature column
- In the example on the previous slides we could convert the Department feature into three new binary features: Computing, Biology, and Chemistry
  - Binary values can then be used to indicate the presence of a particular department
  - For example, a Computing sample would be encoded as Computing = 1, Chemistry = 0, and Biology = 0

Dummy variables

```
import pandas as pd

seriesA = pd.Series(['A', 'C', 'B'])
seriesB = pd.Series([21, 18, 19])
seriesC = pd.Series([4, 1, 1])
seriesD = pd.Series(['Computing', 'Biology', 'Chemistry'])

df = pd.DataFrame({'Grade' : seriesA, 'Age' : seriesB, 'DegreeYear'
: seriesC, 'Department' : seriesD})
print(df)


grade_mapping = {'F':0, 'D':1, 'C':2, 'B':3, 'A':4}
df['Grade'] = df['Grade'].map(grade_mapping)

print(df)
df = pd.get_dummies(df, columns=["Department"])
print(df)
```

One-hot Encoding

- The disadvantage of one-hot encoding is that if there are a very large number of distinct values for a feature this can consequently mean that we end up with a very large number of additional features.

- One approach that is often used to help mitigate the impact of this is dimensionality reduction (techniques such as PCA), which can allow us to in some cases dramatically reduce the overall number of features.

One-hot Encoding

- How this encoding affects distances?
  - Hamming
  - Manhattan
  - Euclidian

Bag of Words – Scikit learn

Bag of words

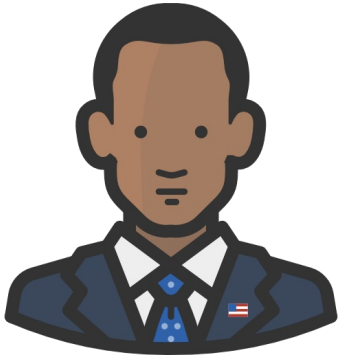
- Used to transform a text into a vector.
- A **bag** is a collection of objects with one property:
  - Order is not important,
    - e.g. {a, b, c} = {a, c, b}
  - Duplicates are allowed,
    - e.g. {a, b, c} != {a, a, c, b}

Bag of words

- We will represent each document by a bag of words.
- This will lose us lots of information. Start thinking about what we will lose!

Running example

- Suppose our dataset contains just these three documents.
- Three tweets from Barack Obama, quoting Nelson Mandela



Running example

- Suppose our dataset contains just these three documents.
- Three tweets from Barack Obama, quoting Nelson Mandela

Tweet 0

No one is born hating another person because of the color of his skin or his background or his religion.

Tweet 1

People must learn to hate, and if they can learn to hate, they can be taught to love.

Tweet 2

For love comes more naturally to the human heart than its opposite.

Tokenization

- First, we must **tokenize** each document. This means splitting it into **tokens**.
- In our simple treatment, the tokens are just the words, ignoring punctuation and making everything lowercase.
- In reality, tokenization is surprisingly complicated,
  - e.g. is "don't" one token or two or three?
  - e.g. maybe pairs of consecutive words (so-called 'bigrams') could also be treated as if they were single tokens ("no one", "one is", "is born")
  - and so on.

Stop-words

- Optionally, discard **stop-words**:
- Stop-words are common words such as "a", "the", "in", "on", "is, "are",…
- Sometimes discarding them helps, or does no harm, e.g. spam detection.
- Other times, you lose too much, e.g. web search engines ("To be, or not to be").

Stop-words – Scikit learn

```
# FYI here's the list of stop-words that scikit-learn uses.

from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

ENGLISH_STOP_WORDS
```

Running example

- After tokenization and discarding stop-words:

Tweet 0

born hating person color skin background religion

Tweet 1

people learn hate learn hate taught love

Tweet 2

love comes naturally human heart opposite

Stemming or lemmatization

- Optionally, apply **stemming** or **lemmatization** to the tokens.
  - E.g. "hating" is replaced by "hate", "comes" is replaced by "come

Tweet 0	**Tweet 1**	**Tweet 2**
born hate person color skin background religion	people learn hate learn hate taught love	love come natural human heart opposite

Count Vectorization

- Each document becomes a vector, each token becomes a feature, feature-values are *frequencies* (how many times that token appears in that document).

Tweet	backgr ound	born	color	comes	hate	hating	heart	human	learn	love	natural	opposit	people	person	religion	skin	taught
0	1	1	1	0	0	1	0	0	0	0	0	0	0	1	1	1	0
1	0	0	0	0	2	0	0	0	2	1	0	0	1	0	0	0	1
2	0	0	0	1	0	0	1	1	0	1	1	1	0	0	0	0	0

The dimension of these vectors

- Sparsity:
  - Here we have n=17 features (columns). How many will there be in general?
  - Most of the feature-values are zero, hence the matrix is sparse. Why?
- We have the curse of dimensionality.
  - Reduce the number of features by:
    - discarding tokens that appear in too few documents (min_df in scikit-learn);
    - discarding tokens that appear in too many documents (max_df);
    - keeping only the most frequent tokens (max_features).

Bag of words - Observations

- It loses all the information that English conveys through the order of words in sentences,
  - e.g. "People learn to hate" and "People hate to learn" have very different meanings but end up with the same bag-of-words representation.
- It loses the information that English conveys using its stop-words, most notably negation,
  - e.g. "They hate religion" and "I do not hate religion" will have the same bag-of-words representation.
- This may not matter for some applications (e.g. spam detection) but will matter for others (e.g. machine translation), for which you need a different representation.

Bag of words – Scikit learn

```
from sklearn.feature_extraction.text import CountVectorizer

tweets = [
"No one is born hating another person because of the color of his skin or his background or his religion.",
"People must learn to hate, and if they can learn to hate, they can be taught to love.",
"For love comes more naturally to the human heart than its opposite."
]

# Create the vectorizer
vectorizer = CountVectorizer(stop_words='english')

# Run the vectorizer
vectorizer.fit(tweets)
X = vectorizer.transform(tweets)
print(X)
```

Bag of words – Scikit learn

- In the example below, we used a CountVectorizer.
- https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

- It does tokenization:
  - By default, it converts to lowercase, it treats punctuation as spaces, and it treats two or more consecutive characters as a word. Each word becomes a token (feature).
- The example discards stop-words.
- The CountVectorizer also, by default, discards any word that appears in every document.

Feature Names

```
# FYI, let's see the tokens (features) that it ends up with
vectorizer.get_feature_names()
```

Vectorize a new document

```
# Vectorize a new document
new_document = "Unsurprisingly, people hate to learn that their
religion loves to hate."

new_document_as_vector = vectorizer.transform([new_document])

# Notice how it ignores words that weren't in the original
tweets, such as "unsurprisingly" and "loves"

print(new_document_as_vector)
```

That's all Folks!
SUPERB wallpapers